

Architecting the Virtual Stochastic Supercomputer: Feasibility, Limitations, and a Blueprint for Next-Generation AI Inference

Zanno Jacklin

Creepybits

2026-07-22

<https://zanno.se>

Executive Summary

The exponential growth in parameters and computational requirements for modern artificial intelligence (AI) has pushed deterministic, high-precision silicon architectures toward their thermodynamic and economic limits. The premise of utilizing a standard consumer host computer as a mechanical "gear" to drive a virtual stochastic supercomputer—one that operates under bespoke mathematical laws to achieve massive energy and computational savings—is a profound architectural hypothesis. By decoupling wall-clock time and physical hardware execution from the virtual inference environment, the concept posits that trading deterministic precision for probabilistic speed could yield order-of-magnitude efficiency gains. This report rigorously evaluates the logic, validity, and physical limitations of deploying a virtual stochastic environment atop conventional host hardware. It identifies a critical bottleneck known as the "emulation tax," wherein the software simulation of non-von Neumann operations (such as stochastic bitstreams) on standard processors results in severe latency and energy penalties. To resolve this, the analysis pivots the paradigm from isolated, physics-defying virtual machines toward hardware-aware, adaptive virtual runtimes. By mapping the proposed "virtual gears" to advanced algorithmic equivalents—including dynamic sub-byte quantization, Logarithmic Number Systems (LNS), temporal feature caching, and entropy-routed hysteresis—the core objectives of the hypothesis can be fully realized without hardware emulation penalties. Finally, a comprehensive technical proposal is presented, outlining the architecture for a Software-Defined Stochastic Inference Engine (SDSIE) capable of executing generative AI workloads with unprecedented efficiency on consumer-grade hardware, paving the way for future integrations with physical neuromorphic substrates.

The Virtual Gear Hypothesis: Theoretical Foundations and Computational Merits

The conceptual foundation of the virtual stochastic supercomputer relies on a "bicycle gear" metaphor¹. In this framework, the physical host processor operates at a low, energy-efficient frequency (acting as the driving cog), while the virtual environment processes multiple computational cycles per host cycle. This is theoretically achieved by altering the fundamental mathematics of the virtual machine, allowing it to bypass the strict binary logic gates of the physical silicon.

Principles of Stochastic Computing

To understand the mechanics of this proposed virtual environment, one must examine Stochastic Computing (SC), a paradigm introduced in the 1950s by John von Neumann². Unlike conventional computing, which relies on positional binary representations (e.g., 32-bit floating-point numbers), SC represents continuous values as probabilities within a stream of random or pseudo-random bits³. For example, the value 0.375 can be encoded as a bitstream where three out of eight bits are '1's (e.g., 00000111)⁴.

The primary advantage of SC lies in its ability to execute complex arithmetic using remarkably simple digital logic, representing a massive reduction in spatial and energetic overhead². In traditional binary computation, a floating-point Multiply-Accumulate (MAC) unit requires an extensive array of shift-registers, carry-lookahead adders, and substantial power budgets⁶. In unipolar SC (where values range from 0 to 1), multiplying two independent bitstreams requires nothing more than a single two-input AND gate². In bipolar SC (where values range from -1 to 1), multiplication is executed via a single XNOR gate². Scaled addition is similarly simplified, requiring only a multiplexer (MUX) driven by a stochastic selection signal².

The Benefits of Probabilistic Workloads

Modern AI, particularly deep neural networks and diffusion models, is inherently probabilistic and vastly over-parameterized⁸. These models do not require the rigid determinism of IEEE 754 standard floating-point arithmetic to approximate functions or predict likelihoods. Simulating a virtual environment that operates on stochastic principles offers substantial theoretical benefits for AI inference:

1. **Extreme Energy Reduction:** By replacing complex binary multiplications with singular logic gate equivalents, the energy consumed per operation drops exponentially. A standard 16-bit MAC unit can consume up to 1.98 milliwatts in a 28nm process, whereas an SC equivalent operates on fractions of a microwatt⁶.
2. **Error Tolerance:** Because data is represented uniformly across a bitstream rather than in a rigid positional hierarchy (where a flipped Most Significant Bit causes catastrophic failure), SC is intrinsically robust against noise. A single bit-flip merely alters the final

probability by a minuscule fraction, providing high fault tolerance³.

3. **Progressive Precision and Early Decision Termination:** SC computes progressively; the output converges toward the true value as the bitstream lengthens⁴. This allows the virtual environment to monitor the convergence latency and execute Early Decision Termination (EDT) if an acceptable accuracy threshold is met before the sequence finishes, yielding dynamic computation times⁴.
4. **Time Dilation:** Decoupling the virtual clock from the host clock allows the virtual environment to advance its internal state by recognizing familiar patterns and skipping redundant calculations. Using memoization techniques akin to the Hashlife algorithm, the virtual system can experience thousands of subjective cycles while the host machine executes a single update¹.
- 5.

The Emulation Tax: Physical and Algorithmic Limitations of Nested Virtualization

While the theoretical advantages of a virtual stochastic supercomputer are profound, implementing this environment purely as a software-defined layer running natively simulated physics on consumer x86 or ARM architecture introduces a fatal bottleneck. This limitation is defined by the hardware architecture and the absolute constraints of Turing completeness: a software simulation cannot extract more computational entropy out of a system than the physical substrate provides¹. This bottleneck is known as the "emulation tax."

The Algorithmic Overhead of Software SC

In a dedicated Application-Specific Integrated Circuit (ASIC), SC logic gates operate instantaneously in parallel. However, simulating SC inside a virtual environment on deterministic hardware forces the host CPU to generate, manipulate, and decode the stochastic bitstreams using sequential software instructions.

To compute an SC operation, the virtual machine must first generate the bitstreams using Stochastic Number Generators (SNGs)⁵. In software, this requires a Pseudo-Random Number Generator (PRNG), such as a Linear Feedback Shift Register (LFSR)⁹. Simulating an LFSR requires multiple clock cycles per bit generated. Once generated, the host CPU must perform bitwise logic operations on the memory registers, followed by a population count (popcount) instruction to determine the resulting probability value¹³.

Operation	Precision Equivalent	Native Hardware Execution	Software Emulation (Virtual Environment)	Latency Penalty

FP32 Binary Multiplication	32-bit	1 Clock Cycle (Dedicated FPU)	N/A	Baseline
Software SC Multiplication	4-bit (16-bit stream)	N/A	~128 Instructions (PRNG + Logic + Popcount)	~128x Slower
Software SC Multiplication	8-bit (256-bit stream)	N/A	~2,056 Instructions (PRNG + Logic + Popcount)	~2,000x Slower
Software SC Multiplication	10-bit (1024-bit stream)	N/A	~8,224 Instructions (PRNG + Logic + Popcount)	~8,000x Slower

The quantitative analysis demonstrates the nesting-doll tragedy of virtual emulation. A standard consumer CPU can execute an FP32 MAC operation in a single pipelined clock cycle¹. To achieve equivalent 8-bit precision in a software SC environment, the host must generate a 256-bit sequence, requiring roughly 2,056 sequential instructions¹². This represents a 2,000-fold slowdown just to emulate what native hardware achieves in a single cycle.

The Roofline Model and Memory Bottlenecks

Furthermore, standard hardware is constrained by the Roofline Model, which dictates that execution speed is limited by either raw compute capability (FLOPS) or memory bandwidth¹. Translating binary tensors into stochastic bitstreams and maintaining the massive contextual state of an AI model within a simulated environment aggressively saturates the memory bus. Data migration across multilevel caches to fuel the software LFSRs triggers thermal throttling and immense energy spikes, entirely defeating the initial premise of saving power¹. Consequently, deploying a literal stochastic emulator on consumer hardware is a mathematical impasse.

Paradigm Pivot: The Hardware-Aware Adaptive Virtual Runtime

To rescue the validity of the virtual gear concept, the implementation must shift from emulating non-existent physical laws to utilizing advanced, dynamic compilation strategies that hook directly into the host's existing hardware accelerators (Tensor Cores, SIMD units, and cache hierarchies)¹.

Instead of an isolated virtual machine simulating stochastic physics, the optimal solution is an **Adaptive Virtual Runtime (AVR)**. This runtime acts as an automated transmission for compute. It dynamically manipulates the precision, resolution, and temporal execution of the AI model based on the complexity of the data, effectively executing the "geared" strategy without incurring the software emulation tax.

This approach successfully maps the theoretical "virtual gears" to state-of-the-art computational optimizations:

- **The Resolution Gear:** Trading precision for speed via Logarithmic Number Systems (LNS) and dynamic sub-byte quantization (e.g., SageAttention).
- **The Temporal Gear:** Compressing time via feature caching and step skipping (e.g., TeaCache, SenCache).
- **The Computational Clutch:** Stabilizing the gear shifts using entropy-routed hysteresis and speculative execution (e.g., Gearbx).

The Resolution Gear: Logarithmic and Quantized Mathematics

Standard consumer processors rely on positional floating-point representations, necessitating vast arrays of power-hungry multipliers. To achieve the 1:10 or 1:100 virtual gear ratio hypothesized, the execution mathematics must be fundamentally altered to replace multiplication with computationally cheaper addition, leveraging the inherent noise tolerance of deep learning models.

The Logarithmic Number System (LNS)

In the Logarithmic Number System, a real number x is represented by its sign s_x and its base-2 fixed-point logarithm l_x , expressed as $x = (-1)^{s_x} 2^{l_x}$ ¹⁵. LNS offers superior performance for evaluating complex non-linear functions because it transposes difficult operations into the simplest arithmetic domains¹⁷.

In the LNS domain, multiplication becomes a straightforward fixed-point addition of the exponents ($L_z = L_x + L_y$), and division becomes subtraction ($L_z = L_x - L_y$)¹⁶. Because neural network dense layers are dominated by multiply-accumulate operations, transforming multiplication into low-latency addition drastically reduces hardware complexity and circuit delay. LNS designs have demonstrated multiplication latencies as low as 4 nanoseconds, compared to 22 nanoseconds for standard floating-point equivalents¹⁵. However, the "loss of torque" in this virtual gear is precision during addition. Logarithmic

addition ($x + y$) is highly non-linear, governed by the exact transcendental function:

$$L_z = \max(L_x, L_y) + \log_2(1 + 2^{-|L_x - L_y|}) \quad 18.$$

Evaluating the Δ term ($\log_2(1 + 2^{-|L_x - L_y|})$) natively requires massive Read-Only Memory (ROM) look-up tables (LUTs) that grow exponentially with bit-width, rendering high-precision LNS additions impractical²⁰. To implement LNS in an adaptive virtual runtime, the system must utilize approximate logarithmic computations. By employing bit-shift approximations derived from first-order Taylor series or piecewise polynomial interpolations, the runtime accepts a controlled loss of deterministic accuracy in exchange for exponential speedups¹⁹. Because neural network gradients naturally absorb uncertainty as noise during backpropagation and inference, the LNS domain acts as a highly efficient high-speed gear²³.

Architecture / Benchmark	Operation Evaluated	Execution Speedup vs. FP32 Baseline
LNS ALU (Hardware)	Vector Dot Product	~2.0x - 2.5x ¹⁵
Shared Approximate LNU	General Non-linear Functions	~2.14x Speedup, ~1.92x Energy Efficiency ¹⁸
Approximate LNS (Software)	Neural Network Training (ResNet)	Comparable Latency, Lower Power Draw ¹⁵

Dynamic Sub-Byte Quantization and MoE Preservation

An alternative to LNS that runs natively on modern GPUs without custom arithmetic logic units is extreme, adaptive quantization. Recent deployments, such as the DeepSeek-R1 671B parameter Mixture-of-Experts (MoE) model compressed to 1.58-bit ternary representations, prove that massive models can maintain structural integrity while operating on fractional bits¹. To operate successfully in a virtual runtime, the quantization cannot be naively uniform.

Quantizing all layers of a massive model to 1.58-bit results in total structural collapse, generating endless loops and nonsensical tokens²⁴. Instead, the runtime implements a dynamic precision map based on architectural sensitivity. For instance, the down_proj layers in the SwiGLU activation gates are hyper-sensitive to quantization because they must scale down the multiplicand of the up and gate projections²⁴. By selectively mapping sensitive early dense layers, embedding matrices, and the LM head to 4-bit or 6-bit precision, while dropping the vast majority of MoE routing weights to 1.58-bit, the virtual runtime shrinks a 720 GB model to 131 GB²⁴. This allows generative execution on consumer hardware at up to 140 tokens per second²⁵.

Outlier Smoothing and SageAttention

When quantizing attention mechanisms—the primary bottleneck in long-context models due to their $O(N^2)$ time complexity—the virtual runtime must deploy specialized kernel techniques such as **SageAttention**²⁶. SageAttention dynamically quantizes the Query (Q) and Key (K) matrices to INT8 or INT4 formats, leveraging high-speed integer Tensor Cores²⁷. However, the numerical range of an INT4 representation is severely restricted to [-7, +7]. If channel-wise outliers exist in the activation data, vital information is scaled down and rounded to zero, completely destroying the model's coherence (e.g., Llama-3 failing MMLU benchmarks at INT4)²⁹. To resolve this, the runtime implements **Outlier Smoothing**. By computing the mean along the token dimension and subtracting it from the tensors, the distribution is centered, effectively flattening the outliers before quantization²⁹.

Furthermore, the runtime utilizes a hardware-specific trick: FP16 accumulators (or FP22 simulated within FP32 registers on Nvidia Hopper architectures) for the $P \times V$ matrix multiplications²⁹. This two-level accumulation prevents precision overflow while doubling the Matmul speed, yielding operations per second (OPS) that surpass FlashAttention2 by up to 3x on RTX 4090 GPUs, effectively putting the attention mechanism into overdrive²⁹.

Attention Kernel	Quantization Level	Target GPU Architecture	Throughput (TOPS) / Speedup
FlashAttention 2	FP16	Ampere / Ada Lovelace	Baseline (165 TOPS) ²⁷
SageAttention	INT8 (QK^T)	Ada Lovelace (RTX 4090)	341 TOPS (~2.1x Speedup) ²⁷

SageAttention 2	INT4 (QK^T), FP8 (PV)	Ada Lovelace (RTX 4090)	481 TOPS (~3.0x Speedup) ²⁹
SageAttention 3	FP4 Microscaling	Blackwell (RTX 5090)	1,038 TOPS (~5.0x Speedup) ³³

The Temporal Gear: Time Dilation via Feature and State Caching

To simulate the effect of the virtual environment moving faster than real-world time, the runtime must decouple from a rigid sequential clock and employ temporal skipping. In generative models, specifically Diffusion Transformers (DiTs), the feature maps generated at timestep t are frequently highly correlated with the features generated at $t - 1$ ³⁴. Forcing the physical host processor to fully recalculate these massive multi-dimensional matrices at every step constitutes a profound thermodynamic waste³⁶.

Timestep Embedding Aware Cache (TeaCache)

TeaCache provides the mechanism to implement the "Temporal Gear." Rather than executing a full forward pass to analyze massive output tensors for redundancy, TeaCache intercepts and evaluates the model inputs, which incurs negligible computational overhead³⁸.

In a diffusion model, the inputs consist of noisy latents, text embeddings, and timestep embeddings. Because the text condition remains constant and the raw noisy latents lack temporal awareness, TeaCache utilizes the *timestep-modulated inputs* (the noisy inputs adjusted by the current timestep embedding)³⁸. The relative L1 distance between consecutive modulated inputs serves as a highly accurate proxy for the difference in final outputs⁴⁰.

The virtual runtime accumulates these L1 distances. Once the accumulated difference crosses a specific polynomial-scaled threshold (e.g., 0.2 to 0.4), the runtime engages the "low gear" and computes the full transformer block⁴¹. If the difference remains below the threshold, the runtime stays in "high gear," skipping the host computation entirely and serving the cached residual from the previous step. This achieves a 1.5x to 2.0x acceleration with negligible loss in visual fidelity⁴⁰.

Sensitivity-Aware Caching (SenCache)

For more volatile or heterogeneous workloads, the runtime escalates to SenCache, which models the denoiser's local sensitivity mathematically⁴². Empirical analysis reveals that relying on a single scalar heuristic (like raw feature magnitude) leads to trajectory deviations under high acceleration³⁹.

SenCache evaluates sensitivity using first-order derivatives (Jacobian matrices). It predicts the output deviation based jointly on the latent drift ($\|\Delta x_t\|$) and the temporal spacing ($\|\Delta t\|$)⁴². By explicitly calculating the network's derivative response to input perturbations, SenCache mathematically bounds the caching error. If the predicted deviation falls below a defined tolerance ϵ , the cache is reused⁴². This dual-proxy approach allows the virtual environment to aggressively fast-forward through predictable generation states, significantly outperforming magnitude-based heuristics (MagCache) in structural integrity and prompt adherence⁴².

Caching Strategy	Target Model	Latency/NFE Reduction	Visual Fidelity Retention (PSNR / SSIM)
Baseline (No Cache)	Wan 2.1 (T=50)	50 NFE	30.60 / 0.937 (Baseline) ⁴³
TeaCache (Fast)	Wan 2.1 (T=50)	25 NFE (50% Cache)	25.06 / 0.869 ⁴³
MagCache (Fast)	Wan 2.1 (T=50)	21 NFE (58% Cache)	28.36 / 0.914 ⁴³
SenCache (Fast)	Wan 2.1 (T=50)	21 NFE (58% Cache)	29.14 / 0.921 (Superior Quality) ⁴³

The Computational Clutch: Entropy-Routed Hysteresis and Speculative Execution

A dynamic, multi-gearred virtual environment is inherently unstable without a computational clutch. If an AI prompt's complexity oscillates, the runtime might rapidly shift between 4-bit and 16-bit precision, or between cached and computed states. The overhead of swapping weight modules from CPU RAM to GPU VRAM, reallocating memory pointers, and reconfiguring the linear modules would stall the host. This phenomenon is known as the "gear hunting" problem¹.

To solve this, the virtual runtime utilizes **Entropy-Routed Dynamic Quantization**, a concept heavily inspired by systems like Gearbx⁴⁴.

Shannon Entropy as a Routing Metric

Not all tokens are equally difficult to generate. Syntax, conjunctions, and predictable formatting have low uncertainty, while complex mathematical reasoning, rare vocabulary, or creative transitions produce broad, highly uncertain distributions⁴⁴. Static quantization forces a global compromise, applying one bit-width to every token, thereby wasting precision on filler and degrading quality on hard logic⁴⁴.

The virtual runtime assesses uncertainty in real-time by calculating the Shannon entropy of the output logit distribution at step t :

$$H_t = - \sum_{i=1}^V p_i \log_2 p_i$$

Where p_i is the softmax probability of the i -th token in vocabulary V , and the theoretical bounds range from $0 \leq H_t \leq \log_2 V$ ⁴⁴. If H_t is low, the model is confident, and the

runtime shifts into high-speed 4-bit precision (the 10th gear). If H_t spikes, indicating confusion, the runtime physically downshifts into FP16 (the 1st gear) to apply maximum deterministic precision⁴⁴.

The Schmitt Trigger (Hysteresis)

To prevent chattering and gear-hunting when the entropy hovers near a threshold boundary, the runtime utilizes a dual-threshold hysteresis buffer, analogous to a Schmitt trigger in electronic engineering¹.

The system defines two boundaries during a calibration phase: an upper threshold θ_{high} and a lower threshold θ_{low} . Crucially, it incorporates a margin of hysteresis (h)⁴⁴.

- The system only upshifts from 8-bit to 4-bit if the rolling-window average of the entropy drops strictly below $\theta_{low} - h$.
- Once engaged in 4-bit, it will not downshift back to 8-bit until the entropy rises above $\theta_{low} + h$ ⁴⁴.

By enforcing a minimum duration for each gear state and utilizing predictive pre-warming

(analyzing the entropy slope ∇H to load upcoming modules into cache ahead of time), the runtime absorbs volatile spikes. This physical module replacement mechanism acts as a true automated transmission, constantly optimizing the arithmetic intensity to ensure the host processor maximizes computational throughput against memory bandwidth constraints⁴⁴.

Speculative Decoding

To further decouple execution time from clock time, the runtime employs Speculative Decoding. Rather than generating every token sequentially through the massive, slow target model, the runtime uses a highly compressed, rapid "draft" model to propose a sequence of candidate tokens⁴⁸.

Because parallel evaluation on modern GPUs is significantly faster than autoregressive generation, the massive target model verifies the entire block of drafted tokens in a single forward pass⁴⁹. Tokens are accepted using a mathematically lossless rejection sampling algorithm, ensuring the final output perfectly matches the target model's distribution⁵⁰. If a token is rejected, the model falls back to standard generation from the point of divergence⁵¹.

By eliminating inter-token sequential latency, Speculative Decoding provides immense throughput acceleration, particularly for structured tasks like code generation where acceptance rates are exceptionally high⁵⁰.

Navigating the Memory Wall: Approximate Value Reconstruction

Despite computational optimizations, the ultimate bottleneck in consumer hardware is the memory wall. Moving massive tensors between DRAM and the processor die consumes immense energy and dictates the upper limit of performance⁵³. To realize the virtual stochastic supercomputer, the runtime must interface with the host memory controller using Approximate Computing techniques⁵³.

L2C and MemSZ Architectures

Developed by Ioannis Sourdis and colleagues, architectures like Approximate Value Reconstruction (AVR) and Memory Squeeze (MemSZ) introduce lossy, general-purpose memory compression to the memory hierarchy⁵⁶. Recognizing that deep learning applications can tolerate approximations in large fractions of their datasets, these systems compress memory blocks using low-latency downsampling that exploits similarities between neighboring values⁵³.

The L2C scheme is a hybrid approach that applies aggressive lossy compression to approximation-tolerant data while routing critical data through a lossless compressor⁵⁸.

- **Co-location and Lazy Evictions:** The architecture co-locates compressed memory blocks and uncompressed cachelines in the Last Level Cache (LLC), significantly increasing effective cache capacity⁵³. It employs lazy evictions to avoid the transfer and

processing overheads associated with costly cache flushes⁵⁶.

- **Error Control:** The system utilizes a two-layer error control mechanism, keeping track of life-time accumulated error to ensure the lossy downsampling does not exceed a user-defined threshold (e.g., maximum 1.2% application error)⁵³.

By implementing parallelized Squeeze (SZ) algorithms in hardware, these technologies achieve compression ratios up to 16:1⁵³. This reduces memory traffic by up to 81%, execution time by 62%, and total system energy costs by 25% compared to uncompressed baselines, effectively widening the memory bus without physical hardware alterations⁵³.

Technical Proposal: Blueprint for the Software-Defined Stochastic Inference Engine (SDSIE)

Based on the synthesis of the aforementioned principles, the concrete solution is the **Software-Defined Stochastic Inference Engine (SDSIE)**. This system circumvents the fatal emulation tax by abandoning exact hardware simulation. Instead, it functions as a highly intelligent intercept layer between the AI model and the physical host hardware, orchestrating resources to achieve massive energy reduction and performance acceleration on consumer hardware.

System Architecture Pipeline

The SDSIE operates through a unified pipeline encompassing five primary subsystems:

1. The Speculative Draft Orchestrator

- **Mechanism:** Upon receiving a prompt, the orchestrator routes the request to a highly quantized (1.58-bit) draft model. The draft model rapidly generates a sequence of candidate tokens (e.g., lookahead length $K = 5$).
- **Execution:** The candidates are passed to the target model for parallel verification. The target model utilizes rejection sampling to validate the block, maintaining lossless output fidelity while drastically reducing sequential memory fetches⁴⁹.

2. The Temporal Dilation Engine

- **Mechanism:** For spatial-temporal tasks (Diffusion Models/Video Generation), the Dilation Engine intercepts the standard forward pass.
- **Execution:** It calculates the L1 relative distance of the timestep-modulated inputs (TeaCache) and the Jacobian sensitivity vectors (SenCache)⁴⁰. If the predicted deviation falls below the tolerance threshold ϵ , the engine entirely bypasses the transformer blocks, injecting the cached residual from the previous step into the normalization layer,

effectively skipping time and saving full computational blocks⁴¹.

3. The Entropy Router & Hysteresis Controller

- **Mechanism:** Operating as an asynchronous background daemon, the Entropy Router continuously monitors the logit distribution of the target model's outputs. During the initial prefill stage, it calibrates the baseline entropy limits (θ_{high} and θ_{low})⁴⁴.
- **Execution:** Utilizing a Schmitt trigger hysteresis buffer, the controller dictates the precision of the upcoming autoregressive layers⁴⁴. If entropy spikes (indicating confusion), it routes execution to FP16 modules. If entropy stabilizes (high confidence), it physically swaps the nn.Linear modules with packed INT4 buffers, offloading high-precision weights to the CPU and shrinking the VRAM footprint dynamically⁴⁴.

4. The Dynamic Resolution Kernels

- **Mechanism:** To execute the mathematics, the SDSIE relies on a library of Just-In-Time (JIT) compiled Triton kernels optimized for consumer GPU architectures (e.g., Ada Lovelace, Ampere).
- **Execution:** For attention mechanisms, it deploys SageAttention2 protocols²⁹. The kernels automatically apply per-thread quantization scaling and subtract token-wise means to smooth channel outliers³⁰. They utilize FP22/FP16 accumulators to prevent precision overflow while doubling Matmul throughput, maximizing integer Tensor Core utilization²⁷.

5. Approximate Memory Subsystem

- **Mechanism:** At the lowest level, the SDSIE interfaces with the host memory controller using L2C/MemSZ paradigms⁵⁷.
- **Execution:** Less critical activation tensors are aggressively downsampled via lossy compression, while sensitive structural weights are losslessly compacted. This widens the effective bandwidth and slashes the thermodynamic cost of memory I/O⁵³.

The Physical Horizon: Emerging Neuromorphic and Stochastic Hardware

While the SDSIE represents a deployable software architecture for today's conventional silicon, the ultimate realization of the virtual stochastic supercomputer requires transitioning these concepts back into physical hardware designed specifically for non-von Neumann computing³.

FPGA Emulators and Probabilistic Spin Logic

To overcome the software emulation tax of PRNG generation in traditional SC, researchers are actively developing Field-Programmable Gate Array (FPGA) digital emulators for probabilistic

bits (p-bits)⁶². A p-bit is a binary unit that fluctuates between states based on simulated thermal noise governed by an activation function⁶². By implementing parametrized p-bit architectures directly on FPGAs, large-scale stochastic networks operate in fully parallel, asynchronous modes without the instruction-scheduling overhead of standard CPUs or GPUs⁶². These systems natively support invertible Boolean logic and Boltzmann machine dynamics, executing complex probabilistic optimizations and Bayesian inferences instantaneously⁶³.

Analog In-Memory Computing and StoX-Net

A primary bottleneck in deploying crossbar-based analog In-Memory Computing (IMC) for Deep Neural Networks is the Analog-to-Digital Converter (ADC) overhead, which consumes 60% to 80% of the hardware's energy and area⁶⁵. Emerging architectures like StoX-Net eliminate this bottleneck by leveraging Spin-Orbit-Torque Magnetic Tunnel Junctions (SOT-MTJs)⁶⁵. By processing array-level partial sums via the stochastic switching dynamics of SOT-MTJs, the hardware achieves a 130x improvement in the Energy-Delay-Product compared to full-precision ADCs, bringing stochastic operations directly to the edge⁶⁵.

Ionotronic Memtransistors and 2D Materials

Looking beyond digital logic, the integration of analog memory and compute provides a direct physical substrate for these virtual environments³. Transition Metal Dichalcogenide (TMDC)

materials, such as two-dimensional Tungsten Disulfide (WS_2) and Molybdenum Disulfide (MoS_2), have enabled the creation of ionotronic "memtransistors"⁶⁶.

These devices operate by utilizing the voltage-driven migration of ions (e.g., edge sulfur atoms forming S-S dimers under an applied electric field) to dynamically alter the resistance of the channel, creating analog memory states⁶⁷.

- **Multi-bit Analog Storage:** Recent studies demonstrate that ultra-scaled monolayer WS_2 memtransistors can hold up to 64 distinct non-volatile storage levels (6-bit capability) with high stability at elevated temperatures⁶⁶.
- **Synaptic Plasticity:** These devices intrinsically mimic biological neurons, natively exhibiting excitatory post-synaptic current (EPSC), Spike-Time Dependent Plasticity (STDP), long-term potentiation, and Pavlov's associative learning without requiring digital logic conversion⁶⁸.

By combining these emerging materials with Superparamagnetic Tunnel Junctions (SMTJs) that inherently generate truly random bitstreams via thermal fluctuations⁷², future host computers will physically possess the "stochastic gears" hypothesized. The AI model will not require a

software translation layer; its weights will exist as physical resistance states across a crossbar array, and inference will occur instantaneously via Kirchhoff's circuit laws, entirely eliminating the memory bottleneck and maximizing thermodynamic efficiency⁶⁵.

Conclusion

The proposition to utilize a conventional consumer computer as a mechanical driver for a virtual, stochastic supercomputer encapsulates a necessary paradigm shift in modern computing. Operating a literal physical simulation of alternative logic architectures via software incurs a massive emulation tax that ultimately defeats the goal of energy efficiency, demanding thousands of instructions to process a single probabilistic bitstream. However, by adapting this concept into a highly optimized, dynamically compiled execution environment—the Software-Defined Stochastic Inference Engine (SDSIE)—the core objectives are fully achievable.

By integrating resolution modulation via dynamic INT4 quantization (SageAttention), temporal skipping via timestep sensitivity evaluation (TeaCache/SenCache), execution stability via entropy-routed hysteresis (Gearbx), and approximate memory compression (L2C), a consumer host can successfully emulate the behavior of a multi-gearred computing environment. This approach extracts maximum semantic output from minimal deterministic labor. As the industry advances toward neuromorphic substrates and ionotronic memory arrays, these software-defined principles will seamlessly transition into the foundational firmware of the next generation of artificial intelligence hardware, fundamentally breaking the physical and economic boundaries of current von Neumann architectures.

Works cited

1. Blueprint of a virtual, low energy stochastic supercomputer.md
2. Stochastic Computing Emulation of Memristor Cellular Nonlinear Networks - MDPI, <https://www.mdpi.com/2072-666X/13/1/67>
3. (PDF) Stochastic Computing In Beyond Von-Neumann Era: Processing Bit-Streams In Memristive Memory - ResearchGate, https://www.researchgate.net/publication/359456819_Stochastic_Computing_In_Beyond_Von-Neumann_Era_Processing_Bit-Streams_In_Memristive_Memory
4. Stochastic computing in convolutional neural network implementation: a review - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7924419/>
5. Simultaneous Area and Latency Optimization for Stochastic Circuits by D Flip-Flop Insertion | Request PDF - ResearchGate, https://www.researchgate.net/publication/325725821_Simultaneous_Area_and_Latency_Optimization_for_Stochastic_Circuits_by_D_Flip-Flop_Insertion
6. A Survey of Stochastic Computing in Energy-Efficient DNNs On-Edge - IEEE BigDataSE-18, <https://www.cloud-conf.net/ispa2021/proc/pdfs/ISPA-BDCloud-SocialCom-Sustai>

- [nCom2021-3mkulWCJVSdKJpBYM7KEKW/264600b554/264600b554.pdf](https://arxiv.org/abs/2011.03494)
7. Stochastic Computing Emulation of Memristor Cellular Nonlinear Networks - PMC - NIH, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8779373/>
 8. Low-bit Model Quantization for Deep Neural Networks: A Survey - arXiv, <https://arxiv.org/html/2505.05530v1>
 9. Introduction to Dynamic Stochastic Computing, http://www.ece.ualberta.ca/~jhan8/publications/IEEE_CASM_202007282312.pdf
 10. Stochastic Circuits for Real-Time Image-Processing Applications, https://homes.cs.washington.edu/~armin/DAC_2013.pdf
 11. (PDF) Stochastic Computing Convolution Neural Network Architecture Reinvented For Highly Efficient Artificial Intelligence Workload on Field Programmable Gate Array - ResearchGate, https://www.researchgate.net/publication/377283522_Stochastic_Computing_Convolution_Neural_Network_Architecture_Reinvented_For_Highly_Efficient_Artificial_Intelligence_Workload_on_Field_Programmable_Gate_Array
 12. Parallel Stochastic Computing Architecture for Computationally Intensive Applications, <https://www.mdpi.com/2079-9292/12/7/1749>
 13. [unknown_url](#)
 14. Ioannis Sourdis - DBLP, <https://dblp.org/pid/18/150>
 15. Logarithmic number system - Grokipedia, https://grokipedia.com/page/logarithmic_number_system
 16. Computing with Logarithmic Number System Arithmetic: Implementation Methods and Performance Benefits - University of California, Santa Barbara, https://web.ece.ucsb.edu/~parhami/pubs_folder/parh20-caee-comput-w-lns-arithmetic.pdf
 17. High-Efficiency Logarithmic Number Unit Design based on an Improved Cotransformation Scheme, <https://past.date-conference.com/proceedings-archive/2016/pdf/0174.pdf>
 18. Accuracy and Performance Trade-offs of Logarithmic Number Units in Multi-Core Clusters - Integrated Systems Lab - ETH Zürich, <https://iis-people.ee.ethz.ch/~michscha/papers/arith2016.pdf>
 19. arXiv:1910.09876v1 [cs.LG] 22 Oct 2019, <https://arxiv.org/pdf/1910.09876>
 20. Optimizing Logarithmic Arithmetic on FPGAs - Imperial Spiral, <https://spiral.imperial.ac.uk/bitstreams/1e63439e-7e58-4806-b054-3e7776c1d255/download>
 21. FPGA Designs with Optimized Logarithmic Arithmetic - Department of Computing, <http://www.doc.ic.ac.uk/~wl/papers/10/tc10hf.pdf>
 22. Computing with logarithmic number system arithmetic - University of California, Santa Barbara, https://web.ece.ucsb.edu/~parhami/pubs_folder/parh20-cee-comp-w-lns-arithmetic-final.pdf
 23. Neural Networks Training with Approximate Logarithmic Computations - Medium, <https://medium.com/data-science/neural-networks-training-with-approximate-logarithmic-computations-44516f32b15b>
 24. Run DeepSeek-R1 Dynamic 1.58-bit - Unsloth,

- <https://unsloth.ai/blog/deepseekr1-dynamic>
25. Overview of DeepSeek R1 Dynamic 1.58-bit | npaka - note, <https://note.com/npaka/n/na1126a4e5fda?hl=en>
 26. EmbeddedLLM/SageAttention-rocm - GitHub, <https://github.com/EmbeddedLLM/SageAttention-rocm>
 27. SageAttention: Accurate 8-bit attention for Plug-and-Play Inference Acceleration - arXiv, <https://arxiv.org/html/2410.02367v1>
 28. SageAttention: Accurate 8-bit attention for Plug-and-Play Inference Acceleration - arXiv, <https://arxiv.org/html/2410.02367v9>
 29. ICML Poster SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization, <https://icml.cc/virtual/2025/poster/44114>
 30. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization - arXiv, <https://arxiv.org/html/2411.10958v4>
 31. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization | OpenReview, <https://openreview.net/forum?id=nC8XliUxeg>
 32. SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration - Liner, <https://liner.com/review/sageattention-accurate-8bit-attention-for-plugandplay-inference-acceleration>
 33. NeurIPS Poster SageAttention3: Microscaling FP4 Attention for Inference and An Exploration of 8-Bit Training, <https://neurips.cc/virtual/2025/poster/118711>
 34. Denoising as Path Planning: Training-Free Acceleration of Diffusion Models with DPCache, <https://arxiv.org/html/2602.22654v2>
 35. Compute Only 16 Tokens in One Timestep: Accelerating Diffusion Transformers with Cluster-Driven Feature Caching - arXiv, <https://arxiv.org/html/2509.10312v1>
 36. FastCache: Fast Caching for Diffusion Transformer Through Learnable Linear Approximation - arXiv, <https://arxiv.org/html/2505.20353v3>
 37. No Cache Left Idle: Accelerating diffusion model via Extreme-slimming Caching - arXiv, <https://arxiv.org/html/2512.12604v1>
 38. Timestep Embedding Tells: It's Time to Cache for Video Diffusion Model - arXiv, <https://arxiv.org/html/2411.19108v2>
 39. Timestep Embedding Tells: It's Time to Cache for Video Diffusion Model - ResearchGate, https://www.researchgate.net/publication/386335656_Timestep_Embedding_Tells_It's_Time_to_Cache_for_Video_Diffusion_Model
 40. TeaCache - vLLM-Omni, <https://docs.vllm.ai/projects/vllm-omni/en/latest/design/feature/teacache/>
 41. teacache - vLLM-Omni, https://docs.vllm.ai/projects/vllm-omni/en/latest/api/vllm_omni/diffusion/cache/teacache/
 42. SenCache: Accelerating Diffusion Model Inference via Sensitivity-Aware Caching - Infoscience - EPFL, <https://infoscience.epfl.ch/bitstreams/74a83b1f-d150-4b02-8e99-c421ee7bc3c7/download>

43. SenCache: Accelerating Diffusion Model Inference via Sensitivity-Aware Caching - arXiv, <https://arxiv.org/html/2602.24208v1>
44. Gearbox: Entropy-Routed Dynamic Quantization for LLM Inference - OpenReview, <https://openreview.net/pdf/e792c6c4ed85a12269da4f4a34448096c9508ccb.pdf>
45. DyQ-VLA: Temporal-Dynamic-Aware Quantization for Embodied Vision-Language-Action Models - arXiv, <https://arxiv.org/html/2603.07904v2>
46. TOWARD EFFICIENT LOW-PRECISION TRAINING: DATA FORMAT OPTIMIZATION AND HYSTERESIS QUANTIZATION - OpenReview, <https://openreview.net/pdf?id=3HJOA-1hb0e>
47. Overflow Handling Integrate-and-Fire Silicon-on-Insulator Neuron Circuit Incorporating a Schmitt Trigger Implemented by Back-Gate Effect - PubMed, <https://pubmed.ncbi.nlm.nih.gov/31026933/>
48. Faster, cheaper, just as smart: Improving the economics of LLM inference with speculative decoding - Red Hat, <https://www.redhat.com/en/blog/solving-economics-llm-inference-speculative-decoding>
49. Speculative Speculative Decoding - arXiv, <https://arxiv.org/pdf/2603.03251>
50. DeepSeek quietly fixed the most expensive part of speculative decoding - Medium, <https://medium.com/@mganeshhemanth/deepseek-quietly-fixed-the-most-expensive-part-of-speculative-decoding-83748c55a111>
51. ultimate guide to Speculative decoding - Newline, <https://www.newline.co/@zaoyang/ultimate-guide-to-speculative-decoding--540ed334>
52. Speculative Decoding with vLLM - Hannes Hapke, <https://www.hanneshapke.com/speculative-decoding.html>
53. AVR: Reducing Memory Traffic with Approximate Value Reconstruction - research.chalmers.se, https://research.chalmers.se/publication/512096/file/512096_Fulltext.pdf
54. Approximation and Compression Techniques to Enhance Performance of Graphics Processing Units - research.chalmers.se, https://research.chalmers.se/publication/521610/file/521610_Fulltext.pdf
55. Re-Exam - Chalmerstenta.se, <https://chalmerstenta.se/tenta/DAT278/20210409-inkluderat.pdf>
56. Reducing Memory Traffic with Approximate Compression - research.chalmers.se, https://research.chalmers.se/publication/517203/file/517203_Fulltext.pdf
57. MemSZ: Squeezing Memory Traffic with Lossy Compression - research.chalmers.se, <https://research.chalmers.se/en/publication/521215>
58. L2C: Combining Lossy and Lossless Compression on Memory and I/O - research.chalmers.se, https://research.chalmers.se/publication/528812/file/528812_Fulltext.pdf
59. L2C: Combining Lossy and Lossless Compression on Memory and I/O, <https://research.chalmers.se/en/publication/528812>
60. Hysteresis Activation Function for Efficient Inference - arXiv, <https://arxiv.org/html/2411.10573v3>

61. Towards efficient and reliable artificial intelligence through neuromorphic principles - Royal Society Publishing,
<https://royalsocietypublishing.org/rsta/article/384/2315/20240522/480517/Towards-efficient-and-reliable-artificial>
62. Digital Emulation of p-bits on FPGA-Based Embedded Systems for Spin-Inspired Probabilistic Logic - Preprints.org,
<https://www.preprints.org/manuscript/202601.0807>
63. Efficient CMOS Invertible Logic Using Stochastic Computing - arXiv,
<https://arxiv.org/html/2603.27030v1>
64. Efficient CMOS Invertible Logic Using Stochastic Computing - arXiv,
<https://arxiv.org/pdf/2603.27030>
65. StoX-Net: Stochastic Processing of Partial Sums for Efficient In-Memory Computing DNN Accelerators - arXiv, <https://arxiv.org/html/2407.12378v2>
66. Ionotronic WS2 memtransistors for 6-bit storage and neuromorphic adaptation at high temperature,
https://research.chalmers.se/publication/537520/file/537520_Fulltext.pdf
67. (PDF) Ionotronic WS2 memtransistors for 6-bit storage and neuromorphic adaptation at high temperature - ResearchGate,
https://www.researchgate.net/publication/373926561_Ionotronic_WS2_memtransistors_for_6-bit_storage_and_neuromorphic_adaptation_at_high_temperature
68. Two-Dimensional Semiconductor Nanoribbons for Neuromorphic Computing - Phantoms Foundation,
https://phantomsfoundation.com/GRAPHENECONF/2026/Abstracts/Grapheneconf2026_Mallik_Sameer_Kumar_199.pdf
69. Development of large scale CVD grown two dimensional materials for field-effect transistors, thermally-driven neuromorphic memory, and spintronics applications - ResearchGate,
https://www.researchgate.net/publication/383951073_Development_of_large_scale_CVD_grown_two_dimensional_materials_for_field-effect_transistors_thermally-driven_neuromorphic_memory_and_spintronics_applications
70. Studies of metal oxide and chalcogenide thin film based memristors for memory and neuromorphic computing applications - ResearchGate,
https://www.researchgate.net/publication/392723576_Studies_of_metal_oxide_and_chalcogenide_thin_film_based_memristors_for_memory_and_neuromorphic_computing_applications
71. (PDF) Defect-engineered monolayer MoS2 with enhanced memristive and synaptic functionality for neuromorphic computing - ResearchGate,
https://www.researchgate.net/publication/384072817_Defect-engineered_monolayer_MoS2_with_enhanced_memristive_and_synaptic_functionality_for_neuromorphic_computing
72. Energy-efficient stochastic computing with superparamagnetic tunnel junctions - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7542576/>